

DA5401: DATA ANALYTICS LAB: FINAL PROJECT REPORT

DATA ENGINEERING:

The data consists of two components:

- **Embeddings:** Two sets of embeddings (1024-dimensional vectors) representing medical features. These embeddings are combined to unify for model training.
- **Labels (ICD Codes):** Two sets of label files contain ICD codes associated with each data point. These are read line by line, and ICD codes are split by semicolons and stored as a list.

The embedding vectors are multi-dimensional arrays, and the labels are processed into a multi-hot encoded format (ICD codes are converted into a binary vector where the presence or absence of each code is encoded as 1 or 0, respectively).

Python code:

```
embeddings_1 = np.load(embedding_file_1)
embeddings_2 = np.load(embedding_file_2)
embeddings = np.vstack([embeddings_1, embeddings_2])
labels_1 = load_labels(label_file_1)
labels_2 = load_labels(label_file_2)
labels = labels_1 + labels_2
```

SAMPLING:

- The dataset is split into training and validation sets. The split is not done for test data, as it is provided separately. The validation set is a random sample of 0.85% of the total data. Validation set has been kept small because of two reasons-
 1. **Training on medical data** requires large data to build robust models that can generalize well to new cases. Since medical datasets tend to be limited in size, maximizing training data is important to improve model's performance.
 2. **Small training data** can lead to overfitting, especially when working with complex models which have many parameters. By using more data for training, the model can generalise better.
- The random state has been kept fixed at 42 to ensure that the train-test split is reproducible.

Python code:

```
X_train, X_val, y_train, y_val = train_test_split(embeddings, multi_hot_labels,
test_size=0.0085, random_state=42)
```

MODEL SELECTION:

A dense neural network has been chosen for the data. A neural network was the appropriate choice due to the following reasons-

- The data labels are high-dimensional vectors. Dense neural networks are well-suited to process such high-dimensional data because they can learn non-linear relationships between the input features and the output labels.
- The multiple hidden layers in the NN architecture allow the model to progressively extract hierarchical features from the data, which is particularly beneficial for identifying complex relationships.
- NN are highly flexible and can be adjusted to tailor the architecture and performance to the specific needs of the task.

The model used for this problem consists of-

- **Input layers:** It has 1024 dimensions, corresponding to the embedding size.
- **Hidden layers:** There are two hidden layers with 1024 and 700 units, each followed by a LeakyReLU activation function and batch normalization to stabilize training. LeakyReLU was chosen for faster convergence and better gradient flow, as compared to standard ReLU.
- **Dropout layers:** Dropout layers with a rate of 0.5 have been used to prevent overfitting and improved generalisation.
- **Output layer:** The last layer uses sigmoid activation, for binary outputs.

These hyperparameters are chosen after running models with other parameters and building on the models with best validation scores. Cross validation was not incorporated in the code due to the complex running times of the models.

Python code:

```
base_model_1 = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(1024,)),
    tf.keras.layers.Dense(1024),
    LeakyReLU(negative_slope=0.175),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(700),
    LeakyReLU(negative_slope=0.175),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(1400, activation='sigmoid')
])
```

PERFORMANCE METRICS:

A **F2 score metric** is implemented as the performance metric, as required by the question. The F2 metric is required for high recall values.

Python code:

```
class F2Score(tf.keras.metrics.Metric):
    def __init__(self, name="f2_score", threshold=0.5, **kwargs):
        super(F2Score, self).__init__(name=name, **kwargs)
        self.threshold = threshold
        self.true_positives = self.add_weight(name="tp", initializer="zeros")
        self.false_positives = self.add_weight(name="fp", initializer="zeros")
        self.false_negatives = self.add_weight(name="fn", initializer="zeros")

    def update_state(self, y_true, y_pred, sample_weight=None):
        y_pred = tf.cast(y_pred > self.threshold, tf.float32)
        y_true = tf.cast(y_true, tf.float32)
        tp = tf.reduce_sum(y_true * y_pred)
        fp = tf.reduce_sum((1 - y_true) * y_pred)
        fn = tf.reduce_sum(y_true * (1 - y_pred))
        self.true_positives.assign_add(tp)
        self.false_positives.assign_add(fp)
        self.false_negatives.assign_add(fn)

    def result(self):
        precision = self.true_positives / (self.true_positives + self.false_positives +
        tf.keras.backend.epsilon())
        recall = self.true_positives / (self.true_positives + self.false_negatives +
        tf.keras.backend.epsilon())
        f2_score = (5 * precision * recall) / (4 * precision + recall +
        tf.keras.backend.epsilon())
        return f2_score
```

MODEL TRAINING:

The final model has the following features:

- Adam optimiser: with a learning rate of 0.00009. Adam optimiser helps with automatic adjustment of learning rates, which is useful for complex models. It also helps in faster convergence.
- Binary cross-entropy loss: Binary Cross-Entropy Loss is well-suited for multi-label classification problems because it treats each label independently and calculates the loss for each label using a binary classification approach.
- F2 score
- 150 epochs: It ensured sufficient training iterations for effective learning. It was also easy to monitor convergence of the model, preventing overfitting.
- Batch size of 128: It used for computational convenience and better generalisation.

These hyperparameters are chosen after running models with other parameters and building on the models with best validation scores. Cross validation was not incorporated in the code due to the complex running times of the models.

PERFORMANCE:

EPOCH	TRAINING		VALIDATION	
	F2 Score	Loss	F2 Score	Loss
1	0.0118	0.6015	0.1616	0.0211
10	0.6792	0.0023	0.7314	0.0018
25	0.7727	0.0016	0.7964	0.0015
50	0.8056	0.0014	0.8161	0.0014
75	0.8212	0.0012	0.8209	0.0014
100	0.83	0.0012	0.8266	0.0014
125	0.8374	0.0011	0.8266	0.0014
150	0.8424	0.0011	0.8294	0.0014

F2 score on training data increases steadily over the epochs, indicating significant improvement in the model's ability to achieve correct classifications. Similar argument can be provided for F2 score on validation data. Since the difference in upward movement for the two scores is approximately the same, overfitting is not there.

Validation loss decreases and stabilises to 0.14%, showing that the model's predictions on the validation set are becoming increasingly accurate.